

C++ Lambda Functions

C++ lambda functions are anonymous (unnamed) functions that you can define directly in your code. They are useful for short, simple functions that are used only in specific places, such as with STL algorithms.

Basic Syntax

```
[capture](parameters) -> return_type { body }
```

- **capture:** Variables from the surrounding scope you want to access inside the lambda.
- **parameters:** Function parameters (just like a regular function).
- **return_type:** (Optional) Specifies the return type.
- **body:** The code to execute.

Simple Example

```
auto add = [](int a, int b) {  
    return a + b;  
};  
  
int result = add(2, 3); // result is 5
```

Using Lambda with STL Algorithms

`for_each`

```
#include <algorithm>  
#include <vector>  
#include <iostream>
```

```
int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    std::for_each(v.begin(), v.end(), [](int n){
        std::cout << n << " ";
    });
    // Output: 1 2 3 4 5
}
```

count_if

```
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> nums = {1, 2, 3, 4, 5};
    int count = std::count_if(nums.begin(), nums.end(), [](int i) {
        return i % 2 == 0;
    });
    std::cout << "Even numbers: " << count << std::endl; // pri
}
```

sort

```
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> nums = {5, 2, 4, 3, 1};
    std::sort(nums.begin(), nums.end(), [](int a, int b) {
        return a > b; // sort in descending order
    });
    for (int n : nums) std::cout << n << " "; // Output: 5 4 3 2 1
}
```

remove_if

```
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> nums = {1, 2, 3, 4, 5};
    nums.erase(std::remove_if(nums.begin(), nums.end(), [](int i) {
        return i % 2 == 0; // remove even numbers
    }), nums.end());
    for (int n : nums) std::cout << n << " "; // Output: 1 3 5
}
```

transform

```
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> nums = {1, 2, 3};
    std::vector<int> doubled(nums.size());

    std::transform(nums.begin(), nums.end(), doubled.begin(), [
        return n * 2;
    ]);

    for (int n : doubled) std::cout << n << " "; // Output: 2 4
}
```

find_if

```
#include <vector>
#include <algorithm>
#include <iostream>
```

```
int main() {
    std::vector<int> nums = {1, 3, 5, 8, 11};
    auto it = std::find_if(nums.begin(), nums.end(), [](int n) {
        return n > 5;
    });
    if (it != nums.end())
        std::cout << "First element greater than 5: " << *it <<
}

```

Capturing Variables

Capture by Value

```
int x = 10;
auto printX = [x]() {
    std::cout << x << std::endl;
};
printX(); // prints 10

```

Capture by Reference

```
int y = 20;
auto modifyY = [&y]() {
    y = 30;
};
modifyY();
std::cout << y << std::endl; // prints 30

```

Default Capture

- [=] captures all variables by value
- [&] captures all variables by reference

```
int a = 1, b = 2;
auto f = [=]() { return a + b; }; // both a and b by value
auto g = [&]() { a = 5; b = 10; }; // both by reference
```

Lambda as Function Parameter

```
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> nums = {1, 2, 3, 4, 5};
    int count = std::count_if(nums.begin(), nums.end(), [](int i) {
        return i % 2 == 0;
    });
    std::cout << "Even numbers: " << count << std::endl; // prints 2
}
```

Mutable Lambdas

Lambdas that capture by value are read-only by default. Use `mutable` to allow modification of captured values (but changes only affect the lambda's copy):

```
int z = 5;
auto f = [z]() mutable { z = 100; std::cout << z << std::endl; };
f(); // prints 100
std::cout << z << std::endl; // still prints 5
```

Lambdas with Return Type

You can specify the return type using `->`:

```
auto divide = [](int a, int b) -> double {
    return static_cast<double>(a) / b;
};
```

```
};  
std::cout << divide(5, 2); // prints 2.5
```

Summary Table

Syntax Example	Description
<code>[](){} </code>	No capture, no params
<code>[x](){} </code>	Capture x by value
<code>[&x](){} </code>	Capture x by reference
<code>[=](){} </code>	Capture all by value
<code>[&](){} </code>	Capture all by reference
<code>[x, &y](){} </code>	Capture x by value, y by reference

Lambda functions are powerful for concise, inline functionality in C++. Use them especially with STL algorithms or for quick, one-off operations!